Day 159: Evaluating Recommenders: Precision, Recall, MAP, NDCG, and How Sure

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 6 | Complete study guide

Watch: What are Precision@k and Recall@k? (Inside Learning Machines) | Evaluation Measures for Search and Recommender Systems (James Briggs)

Script: `day159_evaluating_recommenders.py` · every number in this guide is copied by program from `day159_run_output.txt`, the real output of that script.

All week, top-10 lists were judged by precision@10 alone. Today the measures are the subject: precision, recall, average precision, NDCG (also graded by stars) and reciprocal rank are built from scratch, worked by hand, and applied to every model of the week. They agreed on the order of the models; the measures beyond accuracy did not: popularity's lists covered a share of just 0.030 of the catalogue and found none of the less popular films. A paired bootstrap and five fresh splits showed that Day 158's small ‘stars as confidence’ gain is real on graded NDCG (mean difference +0.0142). The sampled-negatives shortcut used by Day 160's paper inflated hit rates about 4.5 times (0.1474 to 0.6638) but kept the models in the same order here.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

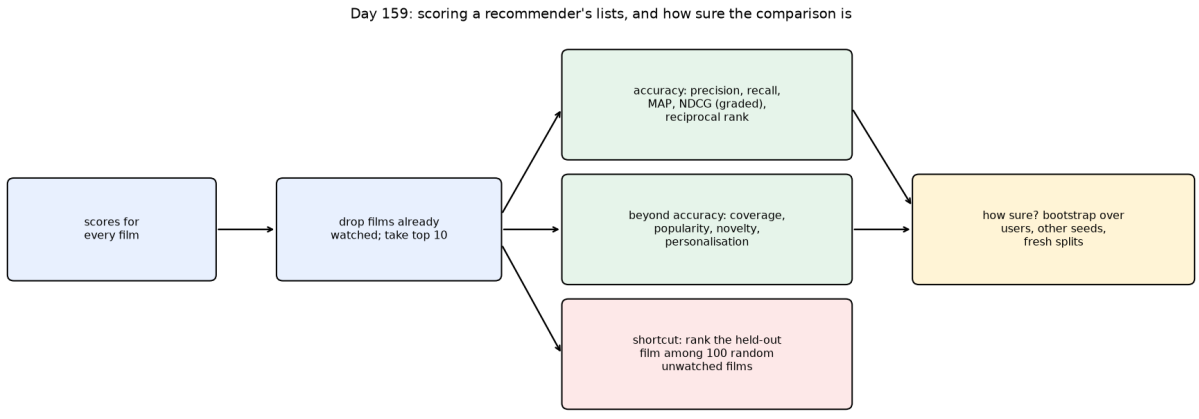
An **offline evaluation** scores a recommender on held-out data: hide some of what users did, ask the model for a list, and count how much of the hidden behaviour the list finds. **Accuracy measures** (precision, recall, MAP, NDCG, MRR) count hits and where they are; **beyond-accuracy measures** (coverage, novelty, personalisation) describe the lists themselves.

A difference between two models is only worth reporting if it would survive other users, another training run and another split. Today measures how sure each difference is.

REAL-WORLD SCENARIO

Two versions of a shop's ‘you might also like’ box are compared on last month's purchases before either is shown to customers. The team must decide which number to trust, whether a 1% difference is real, and whether the winner just recommends the same bestsellers to everyone.

SYSTEM DIAGRAM



Scores become a list; the list is measured for accuracy and beyond it; differences are checked for how sure they are. The sampled shortcut skips ranking every film.

Today's part for the week's build

`day159_exercise.py` is today's parts (per_user_metrics, beyond_accuracy, paired_bootstrap, leave_last_out, sampled_eval), checked by hand.

ACTUAL OUTPUT

```
user 0's top 3 skips watched film 1: [0, 2, 3] (films 0, 2, 3); user 1 has nothing to find, so
    only 1 user is scored
  by hand: hit only at place 2; precision 1/3 = 0.333, recall 1/2 = 0.5, AP (1/2)/2 = 0.25, NDCG
      0.631/(1 + 0.631) = 0.387, first film to find is film 2 at place 2, so RR 0.5
  per_user_metrics: precision 0.333, recall 0.500, AP 0.250, NDCG 0.387, RR 0.500
beyond_accuracy, lists (0, 1), (0, 1), (2, 3); watchers 8, 4, 2, 1, 1 out of 8 users:
  by hand: coverage 4/5 = 0.8; listed films' watchers 8, 4, 8, 4, 2, 1, median 4; novelty
      -log2(watchers/8) = 0, 1, 0, 1, 2, 3, mean 1.167; overlaps 1 (first two users) and 0 twice,
      so personalisation 1 - 1/3 = 0.667
  beyond_accuracy: coverage 0.800, median 4, novelty 1.167, personalisation 0.667
paired_bootstrap: a model against itself +0.000 [+0.000, +0.000], no-gain share 100%; 0.1 better
    for every user +0.100 [+0.100, +0.100], no-gain share 0%
leave_last_out, users 0, 0, 1, 1, 1 at times 5, 3, 1, 9, 9: test rows [0, 4] (row 0, and row 4: a
    tie goes to the later row), training rows [1, 2, 3]
sampled_eval with 10 negatives: target scored highest (film 19): hit rate 1.0, NDCG 1.0; lowest
    (film 0): hit rate 0.0, NDCG 0.0
```

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Four ways to score one list

DEFINITION — start here, no prior knowledge assumed

Precision@k: hits / k. **Recall@k**: hits / films to find. Neither cares where in the list the hits are.

Average precision (AP@k): the precision at each place where there is a hit, averaged; MAP is its mean over users. **NDCG@k**: each hit earns its gain divided by $\log_2(\text{place} + 1)$, and the total is divided by the best possible total. The gain can be 1 for every hit, or graded, here $2^{\text{stars}} - 1$, so a 5-star film is worth 31 and a 1-star film 1. **Reciprocal rank**: 1 / place of the first film to find (MRR averages it).

REAL-WORLD SCENARIO

Two search results pages each contain the answer you want. On one it is first; on the other it is at the bottom. Precision calls them equal; you do not.

MATHEMATICS

$$AP@k = (1 / \min(n, k)) \sum_{\text{places } j \text{ with a hit}} precision@j; \quad RR = 1 / (\text{place of the first hit})$$

$$DCG@k = \sum_{j=1..k} gain_j / \log_2(j + 1); \quad NDCG@k = DCG@k / (DCG@k \text{ of the best possible order})$$

IMPLEMENTATION

```
def per_user_metrics(S, seen, gain, k=10):
    S = np.where(seen, -np.inf, S); held = gain > 0
    who = np.flatnonzero(held.sum(1) > 0)
    order = np.argsort(-S, axis=1, kind="stable")
    top = order[:, :k]; hit = np.take_along_axis(held, top, 1)
    disc = 1.0 / np.log2(np.arange(2, k + 2))
    dcg = (np.take_along_axis(gain, top, 1) * disc).sum(1)
    idcg = (-np.sort(-gain, axis=1)[:, :k] * disc).sum(1)
    nrel = held.sum(1)
    ap = ((np.cumsum(hit, 1) / np.arange(1, k + 1)) * hit).sum(1) / np.maximum(np.minimum(nrel,
        k), 1)
    pos = np.empty_like(order); np.put_along_axis(pos, order, np.arange(S.shape[1])[None,
        :].repeat(len(S), 0), 1)
    rr = 1.0 / (np.where(held, pos, S.shape[1]).min(1) + 1)
    return dict(precision=hit.sum(1)[who] / k, recall=(hit.sum(1) / np.maximum(nrel, 1))[who],
        map=ap[who],
                ndcg=(dcg / np.maximum(idcg, 1e-12))[who], mrr=rr[who]), top[who]
```

ACTUAL OUTPUT

```
3 films to find (0, 1 and 10); top-10 list A has films 0 and 1 in places 1-2, list B in places 9-10:
  list A: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
  list B: [2, 3, 4, 5, 6, 7, 8, 9, 0, 1]
  list A: precision 0.200  recall 0.667  AP 0.667  NDCG 0.765  reciprocal rank 1.000
  list B: precision 0.200  recall 0.667  AP 0.104  NDCG 0.277  reciprocal rank 0.111
by hand, A: precision 2/10; recall 2/3; AP (1/1 + 2/2)/3 = 0.667; NDCG (1 + 0.631)/(1 + 0.631 + 0.5) = 0.765; RR 1
by hand, B: precision 2/10; recall 2/3; AP (1/9 + 2/10)/3 = 0.104; NDCG (0.301 + 0.289)/2.131 = 0.277; RR 1/9
graded NDCG (gain 2^stars - 1): 5-star film first 0.984, 1-star film first 0.640; precision, recall and AP cannot tell them
    apart (0.667 and 0.667)
by hand: ideal 31 + 1 x 0.631 + 1 x 0.5 = 32.131; 5-star first (31 + 0.631)/32.131 = 0.984; 1-star first (1 + 31 x
    0.631)/32.131 = 0.640
Actual output from this lesson's run.
```

What it means

Lists A and B have the same two hits, so precision (0.2) and recall (0.667) cannot tell them apart. Placed first, the hits give AP 0.667 and NDCG 0.765; placed ninth and tenth, AP 0.104 and NDCG 0.277. With graded gains, putting the 5-star film first scores 0.984 and the 1-star film first 0.640, a difference AP cannot see. **First draft:** the first run built list B wrongly (its two hits fell outside the top 10, giving precision 0.1); the hand-worked line beside it showed the mistake.

WHY CHOOSE THIS?

Precision and recall are easy to explain; AP, NDCG and MRR reward putting the best films first, which is what a user scrolling a list experiences.

WHY NOT THIS? (TRADEOFFS)

Each answers a slightly different question, and NDCG's gains and discounts are conventions, not facts.

2. The week's models on every measure

DEFINITION — start here, no prior knowledge assumed

Seven models fitted on Day 154's training split with the settings chosen on earlier days' validation sets: popularity, random, co-occurrence, PureSVD (rank 20), Day 155's explicit ALS, and Day 158's implicit ALS, binary and with stars as confidence. Each user's list ranks every film they have not watched.

REAL-WORLD SCENARIO

Judging runners by finishing time, by average lap or by best lap usually picks the same winner, but not always.

ACTUAL OUTPUT

```
7 models fitted on Day 154's training split (32 s); test: every user's held-out films, every film ranked
                @10  precision   recall     MAP    NDCG  graded NDCG     MRR
         popularity     0.1948   0.1195  0.1215  0.2267       0.1698  0.4380
             random     0.0151   0.0064  0.0051  0.0158       0.0087  0.0592
       co-occurrence     0.2930   0.1948  0.2305  0.3606       0.2806  0.6126
             PureSVD     0.3507   0.2301  0.2830  0.4218       0.3250  0.6648
        explicit ALS     0.0421   0.0189  0.0127  0.0380       0.0382  0.0916
        implicit ALS     0.3672   0.2438  0.3034  0.4423       0.3448  0.6849
 implicit ALS, stars     0.3703   0.2387  0.3108  0.4478       0.3561  0.7002
  order by   precision: implicit ALS, stars > implicit ALS > PureSVD > co-occurrence > popularity > explicit ALS > random
  order by      recall: implicit ALS > implicit ALS, stars > PureSVD > co-occurrence > popularity > explicit ALS > random
  order by         MAP: implicit ALS, stars > implicit ALS > PureSVD > co-occurrence > popularity > explicit ALS > random
  order by        NDCG: implicit ALS, stars > implicit ALS > PureSVD > co-occurrence > popularity > explicit ALS > random
  order by graded NDCG: implicit ALS, stars > implicit ALS > PureSVD > co-occurrence > popularity > explicit ALS > random
  order by         MRR: implicit ALS, stars > implicit ALS > PureSVD > co-occurrence > popularity > explicit ALS > random
Actual output from this lesson's run.
```

What it means

The measures agree almost everywhere: every one puts the two implicit ALS models first, then PureSVD, co-occurrence, popularity, explicit ALS and random. The one swap is recall, where binary implicit ALS (0.2438) edges the star-weighted one (0.2387), which wins on the other five measures.

MRR is generous to popularity (0.4380, more than twice its precision): one early hit is enough, and the most-watched films are a safe first guess for almost anyone.

WHY CHOOSE THIS?

When the measures agree, the choice of measure matters less than the choice of model.

WHY NOT THIS? (TRADEOFFS)

Agreement here is not guaranteed elsewhere: close models (the two implicit ALS variants) can swap places by measure.

3. Beyond accuracy: coverage, popularity, novelty, personalisation

DEFINITION — start here, no prior knowledge assumed

Coverage: the share of the 1,682 films recommended to at least one user. **Median watchers**: how mainstream the listed films are. **Novelty**: the average of $-\log_2$(share of users who watched the film), in bits: higher means rarer films. **Personalisation**: 1 minus the average overlap between two users' lists. **Tail recall@10**: recall counting only held-out films outside the 10% most watched.

REAL-WORLD SCENARIO

A bookshop that only stocks the top 50 bestsellers will often sell you something, but never the book you would not have found yourself.

IMPLEMENTATION

```
def beyond_accuracy(top, pop, nu):
    L = np.zeros((len(top), len(pop))); L[np.arange(len(top))[:, None], top] = 1.0
    ov = L @ L.T / top.shape[1]; n = len(top)
    return dict(coverage=len(np.unique(top)) / len(pop),
                median_pop=float(np.median(pop[top])),
                novelty=float(np.mean(-np.log2(np.maximum(pop[top], 1) / nu))),
                personal=1.0 - (ov.sum() - np.trace(ov)) / (n * (n - 1)))
```

ACTUAL OUTPUT

```
the 170 most-watched films (10% of 1682) hold 42.1% of the held-out test watches
                     coverage  median watchers  novelty (bits)  personalisation  tail recall@10
          popularity     0.030              346            1.43            0.504          0.0000
              random     0.996               19            5.90            0.994          0.0056
      co-occurrence     0.114              247            1.90            0.844          0.0137
             PureSVD     0.233              216            2.15            0.927          0.0436
        explicit ALS     0.171                7            5.93            0.642          0.0050
        implicit ALS     0.237              233            2.09            0.916          0.0474
 implicit ALS, stars     0.199              241            2.03            0.906          0.0338
Actual output from this lesson's run.
```

What it means

Popular films dominate the test set itself: the 170 most-watched films hold 42.1% of the held-out watches, which is why popularity is hard to beat on accuracy.

Popularity recommended only 0.030 of the catalogue, its lists overlap heavily (personalisation 0.504) and it found none of the less popular films (0.0000). Random lists are the opposite: coverage 0.996, almost no overlap, and almost no hits. Of the accurate models, binary implicit ALS had the widest coverage (0.237) and the best tail recall (0.0474). Stars as confidence, better on accuracy, leaned more to popular films: lower coverage (0.199) and tail recall (0.0338).

WHY CHOOSE THIS?

Catches what accuracy hides: a model can score well by recommending the same few hits to everyone.

WHY NOT THIS? (TRADEOFFS)

No single target: more novelty or coverage usually costs some accuracy, and the right balance is a business decision.

4. How sure is a difference?

DEFINITION — start here, no prior knowledge assumed

A **paired bootstrap** resamples the 942 users with replacement 2,000 times and recomputes the mean difference between two models on the same users each time. The middle 95% of those means is the interval.

Two more sources of doubt are checked directly: the same model retrained with another random start (seed 1), and five fresh random 80/20 splits of the data for Day 158's stars-as-confidence model against the binary one.

REAL-WORLD SCENARIO

A new drug looks better in one hospital. Would it still look better with other patients, another batch of the drug, or another hospital?

MATHEMATICS

$d_u = score_A(u) - score_B(u)$; for $b = 1..2000$: draw 942 users with replacement, m_b = mean d; interval = 2.5th to 97.5th percentile of m_b

IMPLEMENTATION

```
def paired_bootstrap(a, b, B=2000, seed=0):
    d = a - b; rng = np.random.RandomState(seed)
    m = d[rng.randint(0, len(d), (B, len(d)))].mean(1)
    return d.mean(), np.percentile(m, [2.5, 97.5]), (m <= 0).mean(), m
```

ACTUAL OUTPUT

```
paired bootstrap over 942 users, 2000 resamples, graded NDCG@10:
          implicit ALS - PureSVD      : +0.0198, 95% interval [+0.0136, +0.0259], resamples with no gain 0.0%
   implicit ALS, stars - implicit ALS : +0.0113, 95% interval [+0.0068, +0.0157], resamples with no gain 0.0%
  implicit ALS, seed 1 - implicit ALS : -0.0023, 95% interval [-0.0058, +0.0015], resamples with no gain 88.4%
         co-occurrence - popularity   : +0.1108, 95% interval [+0.0995, +0.1225], resamples with no gain 0.0%
fresh random 80/20 splits: graded NDCG@10, stars as confidence against binary (Day 158's settings, not re-tuned):
  split seed 10: binary 0.3447, stars 0.3641, difference +0.0195
  split seed 11: binary 0.3517, stars 0.3620, difference +0.0102
  split seed 12: binary 0.3492, stars 0.3623, difference +0.0131
  split seed 13: binary 0.3379, stars 0.3523, difference +0.0144
  split seed 14: binary 0.3512, stars 0.3652, difference +0.0140
  mean difference +0.0142, smallest +0.0102, largest +0.0195
Actual output from this lesson's run.
```

What it means

Implicit ALS beat PureSVD by +0.0198 graded NDCG, interval [+0.0136, +0.0259]. Stars as confidence beat binary by +0.0113, interval [+0.0068, +0.0157]. Retraining the same model with another seed changed it by only -0.0023, an interval that includes 0 ([-0.0058, +0.0015]), so training noise is smaller than both gains.

On five fresh splits the stars model won every time, by +0.0102 to +0.0195 (mean difference +0.0142): Day 158's small gain on liked films holds up for graded NDCG. It is not a gain on every measure: binary was ahead on recall, tail recall and the leave-last-out hit rate below, and whether those differences are beyond noise was not tested here.

WHY CHOOSE THIS?

Cheap (no retraining) and uses the per-user results already computed; fresh splits test what the bootstrap cannot.

WHY NOT THIS? (TRADEOFFS)

The bootstrap treats users as independent and ignores split and training randomness; re-splitting needs retraining.

5. The sampled-negatives shortcut

DEFINITION — start here, no prior knowledge assumed

Ranking every film for every user can be slow on large catalogues. A common shortcut, used in the neural collaborative filtering paper (He and others, 2017) that Day 160 builds on: hold out each user's latest film (**leave-last-out**), sample 100 films they have not watched, and ask where the held-out film ranks among those 101. **Hit rate@10** is the share of users whose held-out film makes the top 10.

The same models are refitted on the leave-last-out split and scored both ways.

REAL-WORLD SCENARIO

Testing a sniffer dog with one hidden parcel among 100 others, instead of a whole warehouse.

IMPLEMENTATION

```
def leave_last_out(u, t):
    order = np.lexsort((np.arange(len(t)), t, u))
    last = order[np.r_[np.flatnonzero(np.diff(u[order])), len(u) - 1]]
    keep = np.ones(len(u), bool); keep[last] = False
    return np.flatnonzero(keep), last
def sampled_eval(S, seen, item, n_neg=100, k=10, seed=0):
    rng = np.random.RandomState(seed); hr, nd = [], []
    for a in range(len(item)):
        cand = np.flatnonzero(~seen[a]); cand = cand[cand != item[a]]
        neg = rng.choice(cand, n_neg, replace=False)
        rank = (S[a, neg] >= S[a, item[a]]).sum()
        hr.append(rank < k); nd.append(1.0 / np.log2(rank + 2) if rank < k else 0.0)
    return np.mean(hr), np.mean(nd)
```

ACTUAL OUTPUT

```
leave-last-out: 943 held-out films, one per user, models refitted (35 s)
                      full ranking: HR@10  NDCG@10   100 sampled: HR@10  NDCG@10   other negatives: HR@10
          popularity               0.0859   0.0449               0.4093   0.2287                   0.4136
              random               0.0053   0.0031               0.0806   0.0381                   0.0870
       co-occurrence               0.1092   0.0563               0.5090   0.2910                   0.5069
             PureSVD               0.1410   0.0708               0.6458   0.3776                   0.6543
        explicit ALS               0.0117   0.0039               0.1750   0.0784                   0.1771
        implicit ALS               0.1474   0.0752               0.6638   0.3865                   0.6638
 implicit ALS, stars               0.1432   0.0724               0.6511   0.3778                   0.6617
  expected for random scores: full ranking 10/1681 = 0.0059 (roughly), sampled 10/101 = 0.0990
  check: five other draws of random scores, sampled HR@10 0.0965, 0.0912, 0.0848, 0.1018, 0.0997 (mean 0.0948)
  order by HR@10, full ranking: implicit ALS > implicit ALS, stars > PureSVD > co-occurrence > popularity > explicit ALS >
      random
  order by HR@10,      sampled: implicit ALS > implicit ALS, stars > PureSVD > co-occurrence > popularity > explicit ALS >
      random
Actual output from this lesson's run.
```

What it means

Every hit rate is far higher with 100 negatives: implicit ALS from 0.1474 to 0.6638, popularity from 0.0859 to 0.4093. Here the order of the models was the same both ways, and sampling other negatives barely moved the numbers (PureSVD 0.6458 and 0.6543). Krichene and Rendle (2020) showed that sampled measures can rank models differently from full ranking, so the order agreeing here is a finding about this data, not a guarantee.

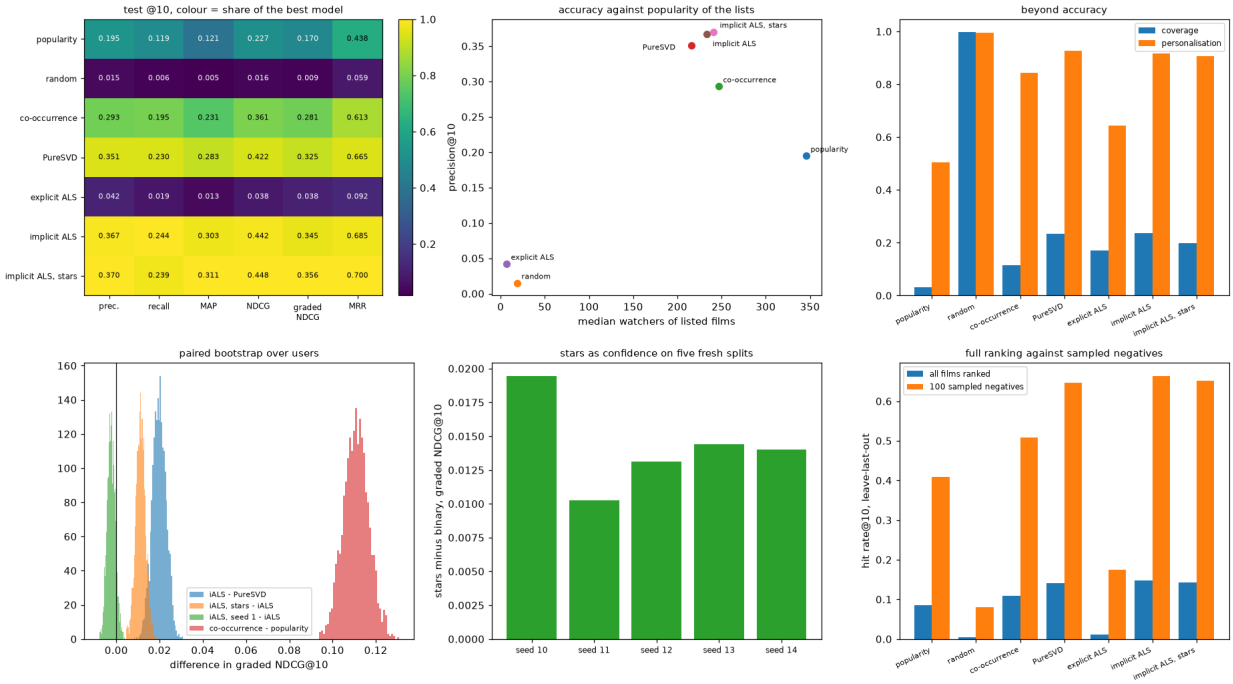
A check: random scores should hit about 10/101 = 0.0990 of the time, but scored 0.0806. Five other draws of random scores averaged (mean 0.0948); the low figure came from one unlucky draw of scores for the held-out films, which resampling the negatives does not change.

WHY CHOOSE THIS?

Fast, and the numbers are comparable with published papers that used it.

WHY NOT THIS? (TRADEOFFS)

Inflated numbers that mean little on their own, and in general it can reorder models; rank every film when you can.



Top: every model on every accuracy measure (colour = share of the best model); precision@10 against how popular the listed films are; coverage and personalisation. Bottom: paired-bootstrap differences in graded NDCG; stars against binary on five fresh splits; hit rate with every film ranked and with 100 sampled negatives.

Interview preparation — technical questions

Q: Precision@k or NDCG@k?

A: Precision counts hits; NDCG also rewards putting them near the top and can weight them by how much the user liked them. Two lists with equal precision here scored NDCG 0.765 and 0.277.

Q: What is MAP?

A: The mean over users of average precision: the precision at each place with a hit, averaged. It also rewards early hits.

Q: How would you check whether model A really beats model B?

A: A paired bootstrap over users on the same test set, retraining with another seed, and fresh splits. Here a seed change moved graded NDCG by -0.0023; stars against binary won on all five fresh splits.

Q: What is catalogue coverage and why does it matter?

A: The share of items recommended to anyone. Popularity covered 3% of the films; a shop using it would never sell most of its stock.

Q: What is the sampled-negatives evaluation and its risk?

A: Rank the held-out item among 100 random ones. It inflates numbers (0.1474 to 0.6638 here) and can reorder models in general.

Interview preparation — theoretical questions

Q: Why do accuracy measures favour popular items?

A: Held-out data is itself dominated by popular items: 42.1% of test watches here went to the 10% most-watched films.

Q: Why use graded relevance?

A: A 5-star film found first is better than a 1-star film found first; binary measures cannot see that, graded NDCG can.

Q: Why might offline gains not show up online?

A: Offline data records only what users did with the old system's recommendations; what they would do with new ones is unobserved. Online A/B tests settle that.

Q: Paired or unpaired comparison?

A: Paired: both models are scored on the same users, so differences between users cancel and smaller gains can be detected.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> A recommender's number depends on what is counted: hits, their places, their stars, or the lists themselves. On this data the accuracy measures agreed on the order of models, while coverage and tail recall showed what popularity costs. A small gain became trustworthy only after a bootstrap, a retrained seed and fresh splits. The sampled shortcut inflated every number but, here, kept the order.

- Implicit ALS: precision@10 0.3672, NDCG@10 0.4423, coverage 0.237.
- Popularity: coverage 0.030, tail recall 0.0000.
- Stars against binary, graded NDCG: +0.0113 [+0.0068, +0.0157]; five fresh splits, mean +0.0142.
- Hit rate@10, implicit ALS: all films ranked 0.1474, 100 sampled negatives 0.6638.

Suggested watch order, if starting from zero

- 1. What are Precision@k and Recall@k? (Inside Learning Machines) — a short explanation of precision@k and recall@k for recommenders.
- 2. Evaluation Measures for Search and Recommender Systems (James Briggs) — the offline measures (recall@K, MRR, MAP@K, NDCG@K) explained with Python.

Day 160 preview

- **Neural collaborative filtering: the week's build.** A small network (a generalised matrix factorisation and a multilayer perceptron side by side) learns from the same watched / not watched data using the week's parts, and is judged with today's measures, both by ranking every film and by the sampled shortcut its original paper used.